Point-Supervised Segmentation of Remote Sensing Images

Technical assessment report · October 2026

1. Introduction

Labeling every pixel of a satellite image takes a lot of time. Clicking a few points per image is much faster, but then the model gets very little supervision: with 20 points, only 0.002% of the pixels in a 1024×1024 image have a label. In this project I trained a segmentation model using only point labels and compared it with a model trained on full masks.

Following the assessment, I did three things:

1. Implemented the partial cross-entropy loss.
2. Simulated point labels on a remote sensing dataset (LoveDA) and trained a segmentation network with this loss.
3. Ran two experiments: how the number of points affects the results, and whether the focal part of the loss helps.

Short summary: with 100 points per image the model was as good as the one trained on full masks (49.7% vs 49.6% mIoU). The focal term did not help; plain partial cross-entropy worked best.

2. Method

2.1 Partial cross-entropy loss

Normal cross-entropy is computed at every pixel. With point labels most pixels have no label, so I compute the loss only at the labeled pixels and divide by how many there are, as in the formula from the task:

$$\mathrm{pCE} = \Sigma\,(\,\mathrm{FL}(\mathrm{pred}, \mathrm{GT}) \times \mathrm{M}\,) \,/\, \Sigma\,\mathrm{M}$$

M is 1 for a labeled pixel and 0 for the rest. FL is the focal loss, $\mathrm{FL} = -(1-\mathrm{p})^{\gamma} \cdot \log(\mathrm{p})$, where p is the predicted probability of the correct class. When $\gamma = 0$ the focal loss is just normal cross-entropy. The task says "partial cross entropy" but the formula uses focal loss, so I implemented both in one function and compared them in Experiment 2.

Dividing by the number of labeled pixels, and not by all pixels, matters: otherwise the loss would become tiny when there are only a few points and the model would barely learn.

I tested the loss before training. With $\gamma = 0$ it gives exactly the same value as PyTorch's `cross_entropy` with `ignore_index=255`, unlabeled pixels get zero gradient, and a batch without any points does not produce NaN.

2.2 Dataset and point labels

I used LoveDA: 1024×1024 aerial images at 0.3 m resolution from urban and rural areas, with seven classes (background, building, road, water, barren, forest, agriculture). The official test labels are not public, so I used the official validation set as my test set and split the official training set 90/10 into train and validation.

Split	Comes from	Images	Urban / Rural
Train	90% of official train	2,269	1,040 / 1,229
Validation	10% of official train	253	116 / 137
Test	Official validation	1,669	677 / 992

To simulate point labels, I took the full mask of each training image and kept n random pixels (n = 5, 20, 100 or 500). All other pixels are marked as unlabeled. Pixels marked "no-data" in LoveDA are never chosen. The points are sampled once with a fixed seed and saved, so every run uses exactly the same labels. I did not re-sample them every epoch, because that would give the model many more labeled pixels than the budget says.

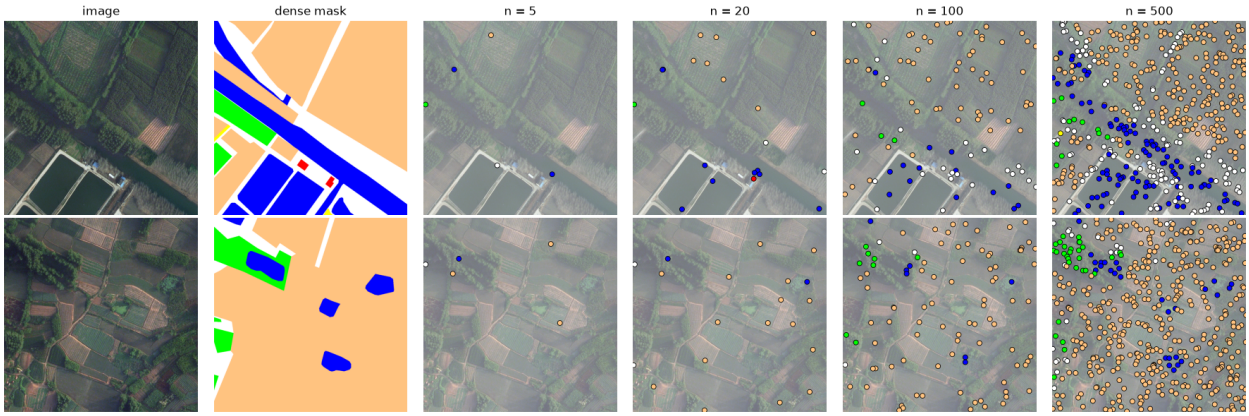


Figure 1. Two training images, their full masks, and the simulated point labels for 5, 20, 100 and 500 points.

One thing to keep in mind: the test set has a different class mix from the training set. For example, agriculture covers 28.6% of the test pixels but 20.1% of the training pixels, and forest 7.4% vs 16.3%.

2.3 Model and training

- U-Net with a ResNet-34 encoder pretrained on ImageNet (segmentation-models-pytorch).
- Random 512×512 crops with flips and 90° rotations. These do not move or blur the points.
- AdamW, learning rate $3{\cdot}10^{-4}$, cosine schedule, 8,000 iterations with batch size 8. The same settings for every run.
- Mixed precision (bf16) on an RTX A2000 8 GB laptop GPU, about 36 minutes of training per run.

2.4 Evaluation

The models are trained only with points but tested on the full masks of the test set at full resolution. I report mIoU (the main metric), overall accuracy (OA) and IoU per class. I always use the model from the last iteration, because in a real point-labeled project there would be no full masks to pick the best checkpoint with. As an upper bound, I also trained the same model on full masks.

3. Experiment 1: how many points do we need?

Purpose. Find out how the number of points per image affects the result, and how close point labels get to full masks.

Hypothesis. More points give better results, but each extra point helps less. Some gap to full masks will still be there at 500 points. Small classes like road will suffer most when there are few points.

Process. I trained five models: 5, 20, 100 and 500 points per image, and full masks. Everything else was the same ($\gamma = 2$, same settings, seed 0).

Results.

Points per image	Labeled pixels	mIoU	% of full masks	OA	Building IoU	Road IoU
5	0.0005%	37.2	75%	60.3	30.0	26.4
20	0.002%	45.3	91%	64.9	52.2	47.5
100	0.01%	49.7	100%	68.2	58.4	51.4
500	0.05%	48.3	97%	67.0	61.0	50.8
Full masks	100%	49.6	100%	68.2	60.1	51.2

Table 2. Experiment 1 results on the test set (%).

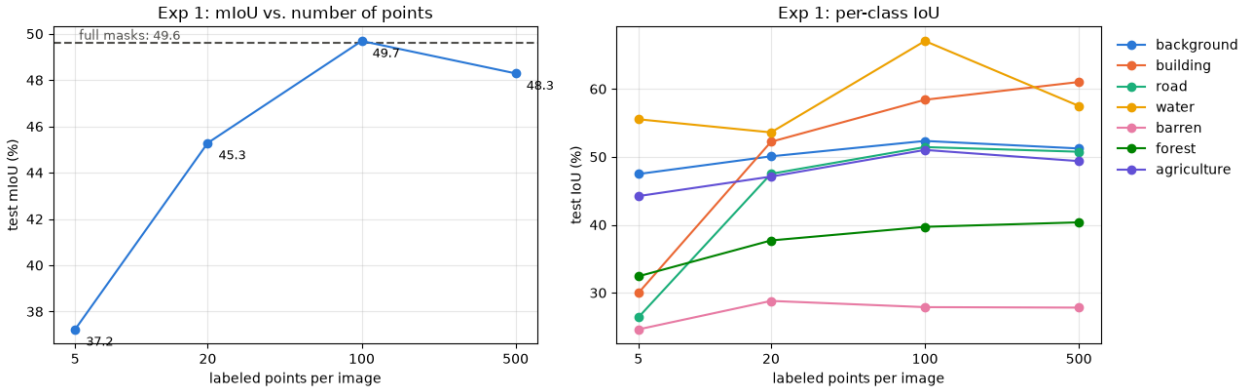


Figure 2. Left: test mIoU against the number of points (dashed line = full masks). Right: IoU per class.

What I found.

- The biggest jump is from 5 to 20 points (+8.1 mIoU). From 20 to 100 it adds 4.4 more, and after 100 it stops improving.
- 100 points per image, only 0.01% of the pixels, already matched full masks: 49.7 vs 49.6 mIoU.
- 500 points was slightly lower (48.3), but with a single run this difference is within noise. Water IoU alone changed by 9.5 points between these two runs.
- With 5 points, buildings and roads suffered most (building 30.0 vs 60.1, road 26.4 vs 51.2). Urban images lost more than rural ones (−16.4 vs −8.5 mIoU), because they have many small objects that few random points hit.

So my hypothesis was right about the diminishing returns and about the small classes, but wrong about a gap at 500 points: there was no gap from 100 points on.

Why does it stop improving? On my validation split, which comes from the same areas as the training images, the models reach about 60% mIoU, but on the test set only about 50%. The test images come from different areas, and more labels from the training areas do not fix that difference.

4. Experiment 2: does the focal term help?

Purpose. The formula in the task uses focal loss. I wanted to check whether it really helps compared to plain partial cross-entropy when the labels are sparse.

Hypothesis. A small γ (1 or 2) gives better results than γ = 0, mainly for rare classes. A large γ (5) is too strong and hurts.

Process. 20 points per image with the same labels for every run, and γ = 0, 1, 2 and 5. Everything else as in Experiment 1.

Results.

γ	mIoU	OA	Background	Building	Barren	Forest
0 (plain partial CE)	47.2	66.7	52.0	53.3	32.0	40.8
1	46.8	66.5	52.2	53.4	30.6	39.1
2	45.3	64.9	50.1	52.2	28.8	37.7
5	42.5	60.5	43.5	45.1	27.7	32.6

Table 3. Experiment 2 results on the test set (%), mIoU, OA and IoU for selected classes.

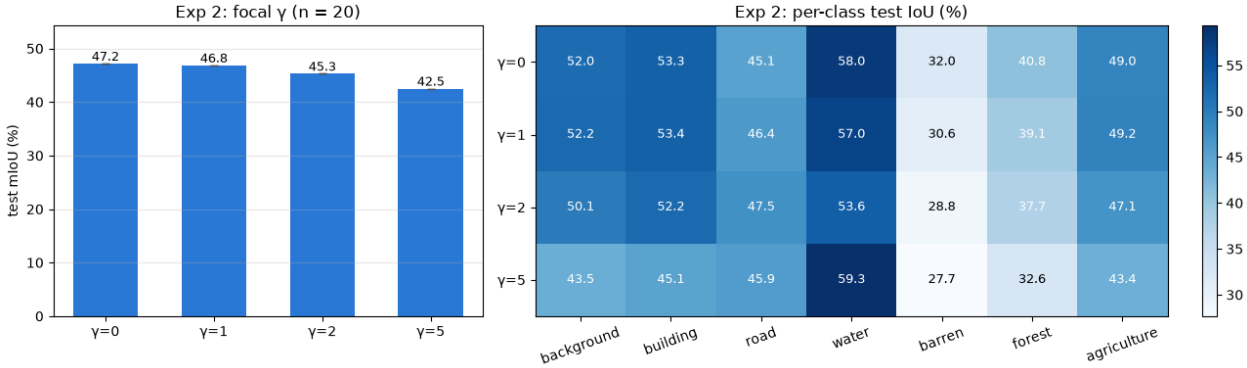


Figure 3. Left: test mIoU for each γ. Right: IoU per class for each γ.

What I found. The higher γ, the worse the result: 47.2, 46.8, 45.3 and 42.5 mIoU. Rare classes did not improve either; barren went from 32.0 down to 28.8 at γ = 2. Only the second part of my hypothesis was right: γ = 5 is clearly the worst.

My explanation: focal loss was designed for object detection, where there are thousands of easy background examples that need to be down-weighted. With only 20 points per image, every point is important. Focal loss lowers the weight of points the model already gets right (at γ = 2, a point predicted with p = 0.9 keeps only 1% of its loss), so the model gets even less signal than it already has. For point labels I would use plain partial cross-entropy (γ = 0).

5. Example predictions

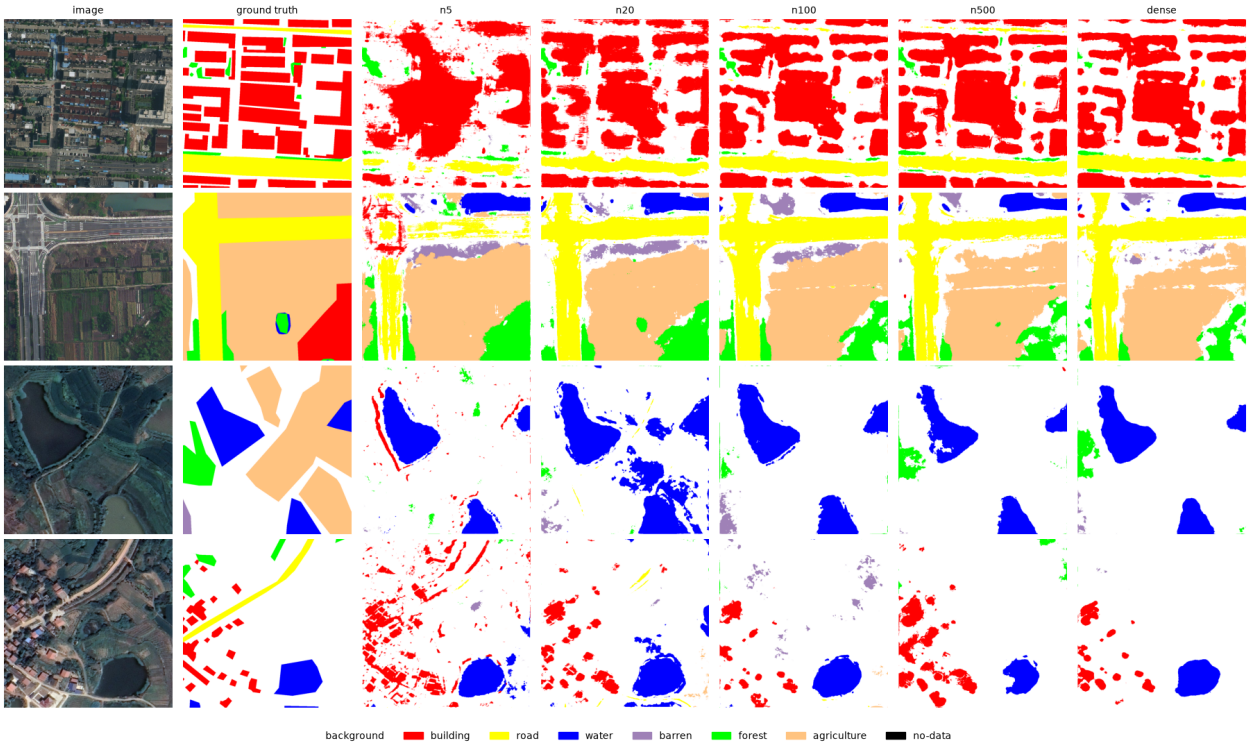


Figure 4. Two urban (top) and two rural (bottom) test images, the ground truth, and the predictions of the Experiment 1 models.

With 5 points the maps are noisy and neighbouring buildings merge into one region. From 20 points on the shapes look right, and from 100 points the predictions look like those of the full-mask model. Some mistakes, like the thin road in the last row, appear in all models, even the one trained on full masks, so they are not caused by the point labels.

6. Limitations and next steps

- I trained each setting only once (one seed), so differences of 1–2 mIoU may be noise. Running three seeds would make the conclusions more reliable.
- Training was short (8,000 iterations) and the validation scores were still going up, so all results would probably improve with longer training.
- Real annotators do not click at random; they usually click in the middle of objects.
- Experiment 1 used $\gamma = 2$, which Experiment 2 showed is not the best choice. The full-mask model used the same loss, so the comparison is still fair.
- The 500-point run crashed at the very end (the data loaders ran out of memory on Windows) and was resumed from its checkpoint at iteration 6,000.

Next, I would use the unlabeled pixels with semi-supervised learning (for example pseudo-labels), try histogram matching to reduce the difference between areas, and test test-time augmentation and ensembles.

All the code is in the notebook `point_supervised_segmentation_local.ipynb`. Running it from top to bottom downloads the data, creates the point labels, trains all 8 models and produces the tables and figures in this report.